A3-PS002-TC383

Purpose-Based Consent Recording & Withdrawal Prototype

Arunima Podder
arunimapodder27@gmail.com

Sreejita Dey
sreejitad05@gmail.com

Prof. Partha Chakraborty
Partha.Chakraborty@iem.edu.in

AIORI Hackathon 3

October 13, 2026

Arunima Podder

Problem Description

- Consent systems often record decisions but do not enforce them where data leaves the service.
- Three failure modes: **stale decision** (cached consent survives withdrawal), **purpose creep** (consent for A used for B), **notice drift** (consent stays valid for a notice the user never saw).
- **Objective:** a prototype in which every downstream data request is checked against current consent, the notice version and the requester's authorization, with a verifiable audit trail.
- **Success criteria:** at least 12 lifecycle tests (28 planned), and the demo grant → share → withdraw → denied → notice change.

Background

- **Terms:** Data Principal (user), Data Fiduciary (service), purpose limitation, and notice versioning.
- **Law:** the DPDP Rules, 2025[1] were notified on 13 November 2025. Rules 3 and 5-16 apply after an 18-month transition, which by our calculation is around May 2027. The notice must give a direct link for withdrawing consent, and withdrawal must be as easy as giving it. digital personal data protection rules 2025 notified +2
- **Consent Managers:** Section 6(7) allows a Data Principal to give, manage, review or withdraw consent through a registered Consent Manager. Consent Managers are expected to keep a tamper-proof record of every consent, withdrawal and notice request, which signals the audit standard regulators have in mind. We are **not** building a Consent Manager. Ksandktsaaro
- **Technical gap:** a consent *record* (a database row) is not consent *enforcement* (a control that blocks access).
- **Scope:** consent-based processing only. Processing under other lawful grounds, erasure, retention and breach handling are out of scope.

Literature & RFC Review

Source	What it defines	What we adopt	What it does not cover
Kantara Consent Receipt Spec [2]	A record of authority granted by a PII Principal to a PII Controller, human-readable and representable as JSON. Its v1.1 was published on 25 May 2018.	Receipt-style fields: purpose, timestamp, notice reference	A runtime enforcement point
ISO/IEC TS 27560:2023 [3]	An interoperable, open and extensible structure for recording consent, covering the consent lifecycle. It does not specify an exchange protocol or an exact data structure for exchange.	Lifecycle thinking (grant → withdraw → change)	Enforcement, exchange protocol
ISO/IEC 29184:2020 [4]	Online privacy notices and consent. 27560 builds upon it.	Notice as a versioned, referenced artefact	Access control
RFC 6962 (Certificate Transparency) [5]	Publicly auditable, append-only logs built on Merkle trees. Consistency proofs show the append-only property.	The idea of append-only, verifiable logs, in a simplified hash chain	Not a consent standard. It is a technique we borrow

Scope: we compare standards, not commercial platforms.

Proposed Solution

- **Components:** Consent UI (React); Consent Service ③④ (grants, withdrawals, notice publication); Audit Service ⑤ (export and verify); Policy Enforcement Point (PEP) ②⑥; Mock Data-Sharing API (runs in the same FastAPI app; internal only, token-gated, no public route); Storage Gateway ① (single write path to D1 and the transaction boundary).
- **Stores:** D1 Ledger (append-only, hash-chained, DB triggers abort UPDATE/DELETE), D2 Notices (immutable once published), D3 Requester Registry (API-key hash, allowed purposes), D4 Synthetic Data.
- **D1 event types:** GRANT, WITHDRAW, GRANT_REJECTED, NOTICE_PUBLISHED, ALLOW, DENY.
- **Notice hash:** SHA-256(canonical_json({purpose, version, text})), one per purpose.
- **Grant:** carries the notice_hash_seen. A stale hash returns 409 and is logged as GRANT_REJECTED. A purpose with no published notice is rejected.
- **Withdraw:** always accepted, with no hash check, and idempotent.
- **Allow rule** (anything not explicitly allowed is denied): requester authenticated, purpose allowed for the requester, notice published, latest consent event is GRANT, and its notice hash equals the current hash.
- **Reason codes:** REQ_AUTH, REQ_PURPOSE, NO_NOTICE, NO_CONSENT (never consented or unknown user, identical response), CONSENT_WITHDRAWN, NOTICE_STALE, ERROR.
- **Audit:** entry_hash = SHA-256(prev_hash ‖ canonical_json(entry)). Every outcome is logged unless the ledger itself is unavailable. Export returns the entries plus the head hash. A standalone script verifies an export without the server.
  User sees own audit entries plus a server-reported check; full verification needs the export (auditor).
- **Error policy:** roll back, discard fetched data, return 403, then a best-effort DENY entry in a fresh transaction. If that fails, write to a local error log.

Optimization Proposed

Badges	Typical weak design	This design
①	Mutable consent row, many write paths	Append-only, hash-chained ledger; single write path; triggers block UPDATE/DELETE
②	Consent checked once, or cached	Checked on every request, no cache, in one transaction
③	Notice stored as free text	Per-purpose notice hash bound to each grant
④	Stale UI binds consent to unseen text	Hash-seen check at grant (409 on mismatch); withdraw never blocked
⑤	Plain editable log	Hash-chained audit with verify endpoint and standalone script
⑥	Errors may fail open	Default deny, fail closed

Design targets.

Solution Architecture & Design

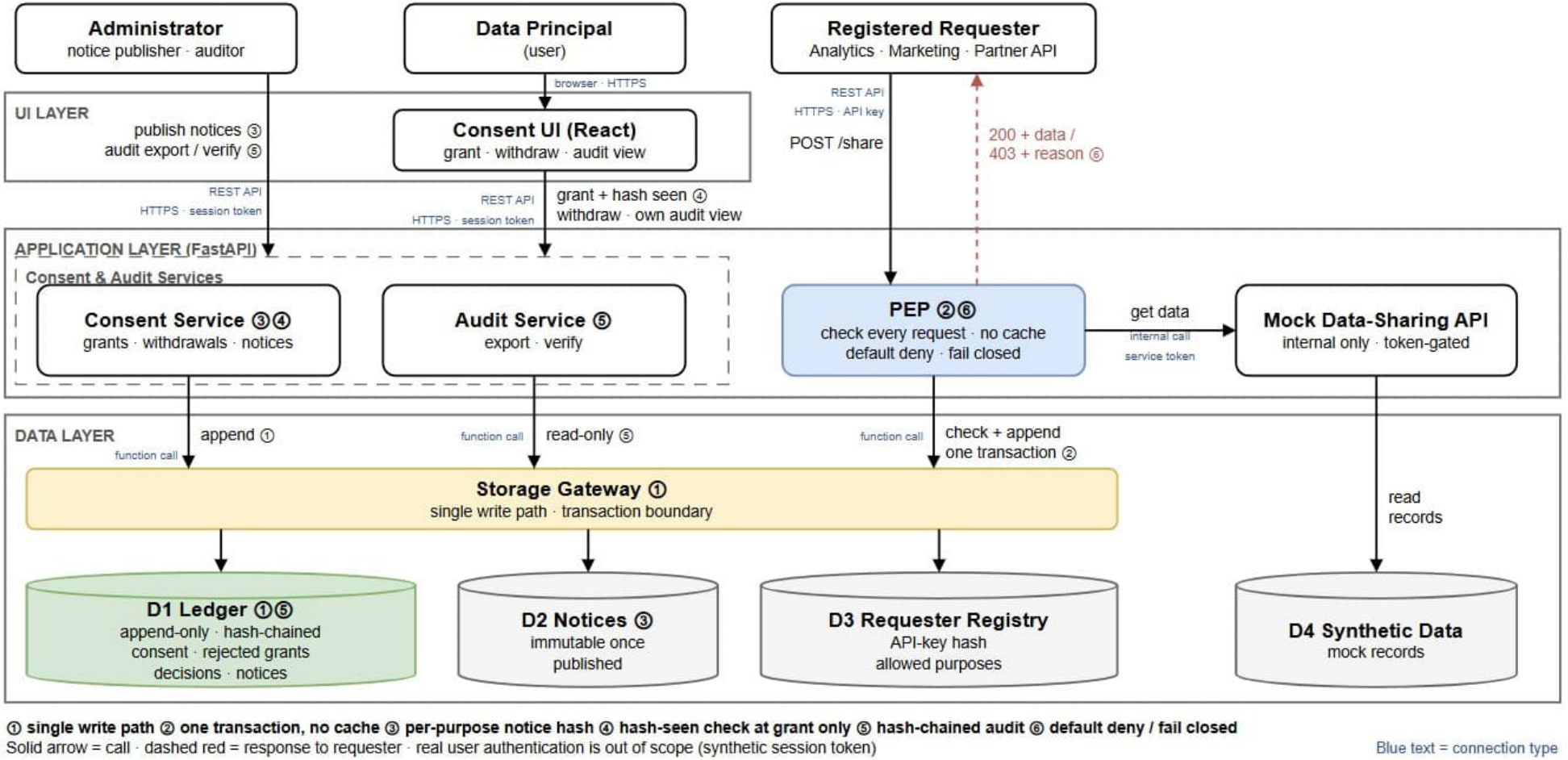


Figure: System Architecture

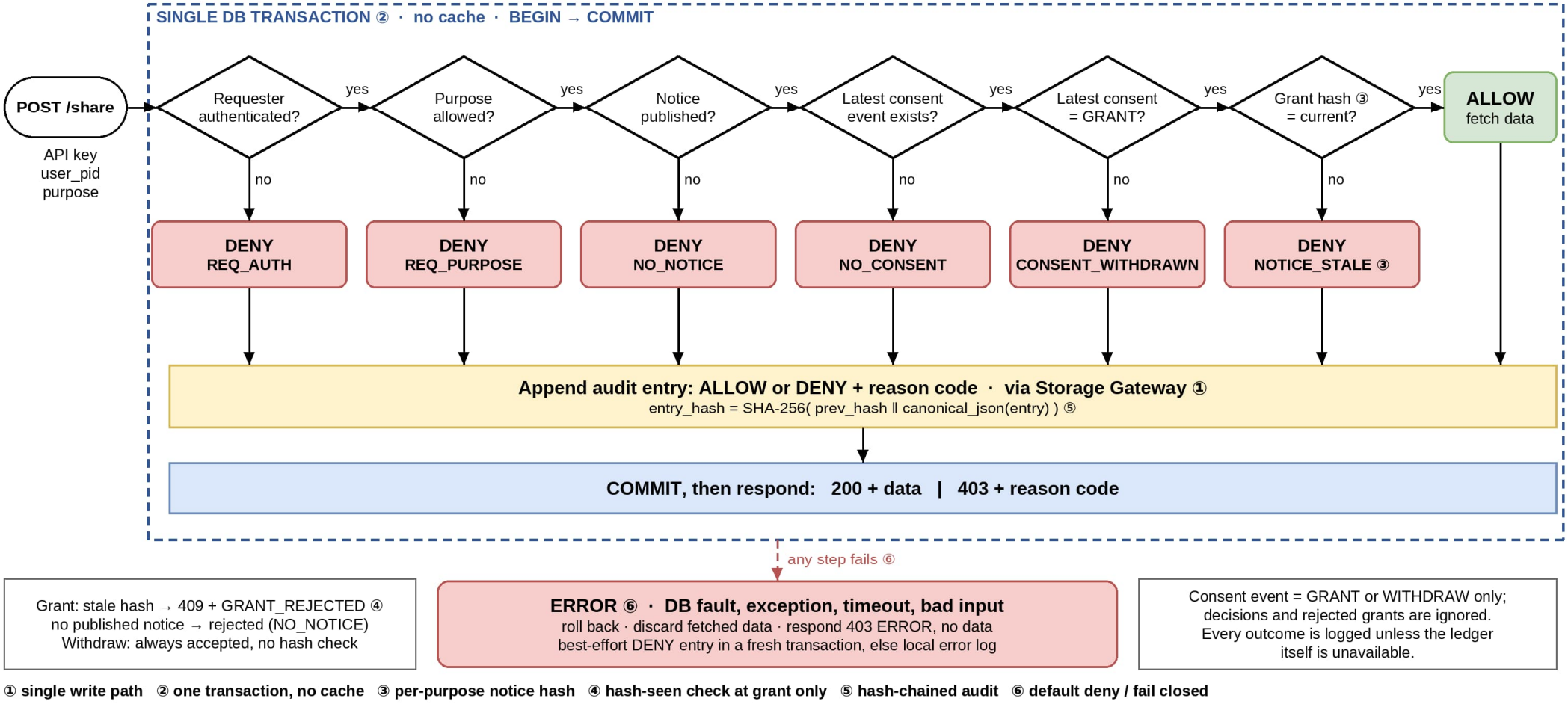
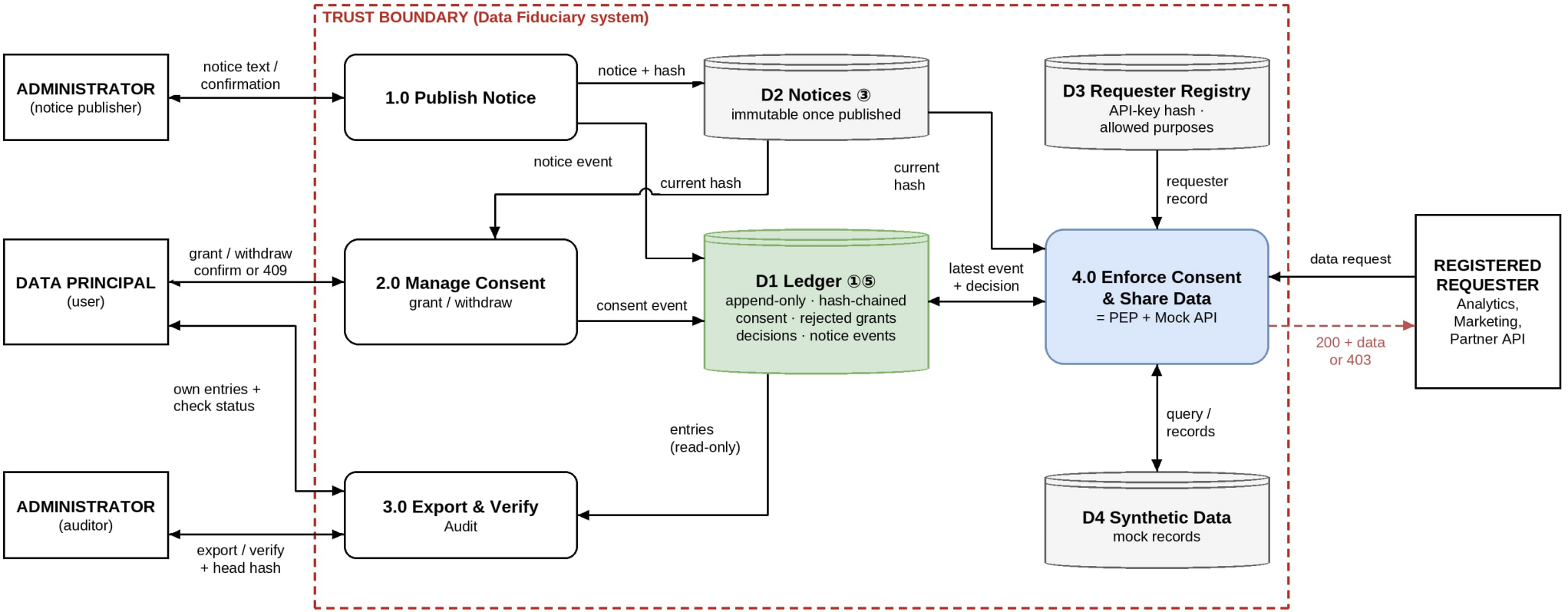


Figure: Enforcement Flowchart

Data Flow Diagram



Rectangle = external entity · rounded box = process · cylinder = data store · arrow = data flow (two heads = request and response) · dashed red = response to requester
Requesters receive data only through process 4.0 (see the enforcement flowchart, slide 8). User sees own entries + a server-reported check; full verification needs the export (auditor).

Figure: Data Flow Diagram (DFD)

Threat Model

Attacker / failure	Control	Residual risk
Requester uses purpose A consent for B	Purpose-bound check and per-requester allowed purposes	Cannot detect a requester lying about actual use
Request after withdrawal	Per-request check, no cache	Data already shared isn't recalled
Request overlapping a withdrawal	Single transaction with the ledger head locked; ledger order is the ground truth	A response already returned stays delivered
Stale UI consent	Hash-seen check at grant	None identified
Log edit, delete or reorder	Hash chain, triggers, standalone verify	Truncation or full rewrite needs an exported head hash
Raw database-file access	Hash chain detects what triggers cannot prevent	Detection only, not prevention
Spoofed requester	Hashed API key	Key theft; mTLS out of scope
Internal fault	Fail closed, best-effort DENY audit, local error log	Possible audit gap if the ledger is unavailable
Reason codes seen by requesters	Unknown user and never-consented are identical	CONSENT_WITHDRAWN and NOTICE_STALE reveal prior consent state (accepted for prototype)
Unauthenticated requests writing DENY entries	Logged with reason code	Ledger growth and lock contention (accepted; rate-limiting is future work)
Mock API call inside the transaction	Same FastAPI app, called in-process with an internal token	Serialises requests (prototype limitation

Out of scope: real user authentication · data erasure and retention · recall of data already shared · real personal data (synthetic only) · legal compliance certification

Evaluation Plan

1. **Correctness:** about 20 lifecycle tests with expected reason codes. Minimum bar is 12.

2. **Concurrency:** many parallel /share calls while one withdrawal commits. Pass if no ALLOW has a ledger sequence after the withdrawal.

3. **Audit tamper detection:** apply edit, delete, reorder and insert to the log and check that verification fails. Test truncation against an exported head hash.

4. **Overhead:** measure PEP latency (p50/p95) with and without the check, and report whatever we measure.

Test List(28):

- *Enforcement:* 1 grant → allow; 2 never consented → NO_CONSENT; 3 unknown user → identical NO_CONSENT; 4 withdraw → CONSENT_WITHDRAWN immediately; 5 re-grant → allow; 6 wrong purpose; 7 unknown requester; 8 requester not authorised; 9 malformed request → 403; 10 no published notice → NO_NOTICE; 11 notice change → NOTICE_STALE until re-grant; 12 notice change on another purpose leaves others valid.
- *Grant and withdraw rules:* 13 stale-hash grant → 409 plus GRANT_REJECTED; 14 rejected grant leaves state unchanged; 15 grant with no published notice rejected; 16 withdraw with stale hash succeeds; 17 withdraw with missing hash succeeds; 18 withdraw with no active consent succeeds.
- *Audit:* 19 history preserved, UPDATE/DELETE blocked; 20 chain verifies when intact; 21 edit detected; 22 delete detected; 23 reorder detected; 24 truncation detected against anchored head; 25 standalone verify on an export file.
- *Robustness:* 26 DB fault → 403, no data; 27 concurrency test; 28 every outcome logged.

Deployment Timeline

Days	Milestone	Done when
1-4	**Foundation and Consent Service:**FastAPI + SQLite, Storage Gateway, hash-chained ledger with triggers blocking UPDATE/DELETE; publish notice (per-purpose hash), grant (hash-seen check, 409 + GRANT_REJECTED), withdraw (always accepted)	Chained entries are written; manual UPDATE/DELETE is rejected; grant, withdraw and notice endpoints pass unit tests
5-7	**PEP + Mock Data-Sharing API:** hashed API-key auth, purpose check, full decision order, reason codes, ALLOW/DENY audit in the same transaction, fail closed	All seven outcomes reproducible via the API; an injected DB error returns 403 with no data
8-9	**Audit Service:** export with head hash, verify endpoint, standalone verify script, user's own-entries view	Edit, delete and reorder are all detected; truncation is detected against an exported head hash
5-11 (parallel)	**React UI**, built against the auto-generated FastAPI API spec: purpose toggles showing notice version, withdraw button, audit view, admin notice page	The full user flow runs from the browser
10-12	**Test suite:** 28 tests, concurrency test, tamper tests, bug fixing	All tests green; no ALLOW after a withdrawal commits
13-14	**Deployment and Freeze :** Docker Compose (one command from a clean clone), seed/reset script, README, scripted demo (grant → share → withdraw → denied → notice change); then bug fixes only	Demo runs end-to-end three times from a fresh start; code frozen

Bibliography

- [1] DPDP Act 2023 and DPDP Rules 2025, MeitY, https://www.meity.gov.in/data-protection-framework (confirm sections from the primary text).
  The Leaflet, “India’s new DPDP Rules, 2025: A Detailed Reading”, https://theleaflet.in/explainer/indias-new-digital-personal-data-protection-rules-2025-a-detailed-reading (secondary).
- [2] Kantara Initiative, Consent Receipt Specification v1.1, https://kantara.atlassian.net/wiki/x/uo01
- [3] ISO/IEC TS 27560:2023, *Privacy technologies, Consent record information structure*, https://www.iso.org/standard/80392.html
- [4] ISO/IEC 29184:2020, *Online privacy notices and consent* , https://cdn.standards.iteh.ai/samples/70331/6d728e6153bb462fb03f476c9d961311/ISO-IEC-29184-2020.pdf
- [5] B. Laurie et al., RFC 6962, *Certificate Transparency*, http://www.rfc-editor.org/info/rfc6962

Thank You for Your Attention!